

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LABORATORY INVESTIGATION & FORENSIC EVIDENCE REPORT

“Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training”

Lab Title: SBT-DF203 Lab 5: ARP Poisoning Forensics	Course Name: Basic Networking Skills for Digital Forensics
Student Name: Almustapha Yusuf	Registration Number: fwsd2511343
Operating Platform: Kali Linux 2026 (Rolling Release)	Date of Submission: 16/09/2026
Investigation Target: ARP Cache Resolution & Poisoning Trace (arp.pcap)	Report Version: Version 1.0 (Final Release)

Executive Summary

This report documents an ARP poisoning investigation following a report that a default gateway intermittently resolved to the wrong MAC address. A normal ARP resolution baseline was captured and analyzed, followed by forensic analysis of the supplied arp.pcap training capture. The analysis identified gratuitous ARP requests from the gateway as the key suspicious indicator, though no classic Ethernet/ARP MAC mismatch was present. Mitigation controls including Dynamic ARP Inspection and network segmentation are recommended.

1. Authorization and Scope

- Authority:** Performed in an isolated Kali Linux VM. No third-party systems targeted.
- Scope:** ARP traffic analysis on a virtualized LAN.
- Live Simulation Note:** Not performed — no second VM available. Documented as a limitation.

2. Network Diagram / Assigned Addresses

The investigation analyzed addressing schemes across two distinct contexts: the active host virtual machine environment and the provided training capture network:

Network Context	Node Identifier	IP Address	Hardware MAC Address
Analyst VM (Live)	Kali Linux (eth0)	192.168.199.128/24	00:0c:29:f3:f7:44
Analyst VM (Live)	Default Gateway	192.168.199.2	00:50:56:ff:50:a0
arp.pcap Capture	Host Client A	136.160.215.15	00:50:56:86:cb:fc
arp.pcap Capture	Host Client B	136.160.215.194	00:50:56:86:02:65
arp.pcap Capture	Target Gateway	136.160.215.1	00:1b:17:00:0a:30

3. Evidence Integrity

Cryptographic SHA-256 hash digests computed for all evidence captures and working duplicates:

Evidence Artifact	SHA-256 Hash Digest	Integrity Status
arp.pcap (Original)	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e	VERIFIED
arp_working.pcap (Working)	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e	VERIFIED
normal_arp.pcapng (Baseline)	431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50	VERIFIED

- **Integrity Conclusion:** All hashes match — evidence is intact.

4. Normal ARP Behaviour

Analysis of normal_arp.pcapng baseline demonstrates standard address resolution operations:

Frame	Type	Sender MAC	Sender IP	Target
1	Request	00:0c:29:f3:f7:44	192.168.199.128	192.168.199.2 (broadcast)
2	Reply	00:50:56:ff:50:a0	192.168.199.2	192.168.199.128

- **Interpretation:** Complete request/reply cycle. Gateway MAC learned and cached.

5. Poisoning Indicators and Timeline

Analysis of the supplied arp.pcap capture file examined IP-to-MAC claims and gratuitous broadcast events:

Claimed IP	Claimed MAC	Count
136.160.215.194	00:50:56:86:02:65	2
136.160.215.15	00:50:56:86:cb:fc	2

Suspicious Probes: Gratuitous ARP Requests Detected:

Frame	Source IP	Source MAC	Query Payload
1	136.160.215.1	00:1b:17:00:0a:30	Who has 136.160.215.1?
7	136.160.215.1	00:1b:17:00:0a:30	Who has 136.160.215.1?

- **Classic Spoofing (eth.src != arp.src.hw_mac):** 0 instances confirmed via comprehensive Python parser.
- **Forensic Interpretation:** The gateway is asking for its own IP — a behavior that can indicate either: (1) A legitimate ARP cache refresh, or (2) An active poisoning attempt where the attacker's presence forced the gateway to re-assert its identity.
- **Absence of MAC Mismatch:** The absence of MAC mismatch means the classic "attacker MAC inserted into ARP payload" pattern was not observed in this capture.

6. ARP-Table Comparison

Clean Neighbor Cache: Clean (Analyst Baseline):

```
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 STALE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE
```

Capture Claims: Poisoned Capture Claims:

```
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
```

7. Impact Assessment

- **Man-in-the-Middle (MITM) Risk:** A successful ARP poisoning attack would allow a man-in-the-middle (MITM) to intercept, modify, or block traffic between the victim and gateway.
- **Capture Severity:** In this bounded capture, the suspicious indicators are limited — no confirmed MITM was observed.
- **Early Warning Indicator:** The presence of gratuitous ARP from the gateway is a potential early warning of an ongoing attack.

8. Restoration and Preventive Controls

Cache Restoration: Restoration Steps (if simulation were performed):

```
sudo ip neigh flush all
ip neigh show
ps aux | grep -E '[a]rp.py|[s]copy'
```

Enterprise Defense: Recommended Preventive Controls:

- **Dynamic ARP Inspection (DAI):** Deploy DAI alongside DHCP snooping on managed access switches to intercept and validate all ARP requests and replies against trusted binding tables.
- **Network Segmentation & VLANs:** Isolate sensitive subnets to limit the blast radius of broadcast domain ARP spoofing.
- **Switch Port Security:** Configure static MAC address binding and port security limits to prevent MAC flooding attacks.
- **Gateway Invariant Monitoring:** Implement network alerting for any sudden change in default gateway IP-to-MAC associations.
- **Enforce End-to-End Encryption:** Utilize HTTPS, TLS, and SSH across all communications so that even if Layer 2 traffic is diverted, payload confidentiality is preserved.
- **Dedicated ARP Sentinel Tools:** Deploy arpwatch, XArp, or IDS rules (Snort/Suricata) to continuously detect anomalous gratuitous ARPs or conflicting replies.

9. Screenshots and Evidence

The following 12 documented figures present the complete laboratory workflow, baseline capture, neighbor table inspections, request/reply field dissections, and Python-driven mismatch analysis:

```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
(kali@kali) [~/SBT-DF203-Lab4]
$ cd ~
mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5

(kali@kali) [~/SBT-DF203-Lab5]
$ pwd
/home/kali/SBT-DF203-Lab5
$ find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali) [~/SBT-DF203-Lab5]
$ sudo apt update
sudo apt install -y wireshark tshark python3-scapy net-tools wget
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [115 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.9 MB in 35s (2,225 kB/s)
2083 packages can be upgraded. Run 'apt list --upgradable' to see them.
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
wget is already the newest version (1.25.0-3).
wget set to manually installed.
The following packages were automatically installed and are no longer required:
gir1.2-girepository-2.0 libcrypt-dev libkmlbase1t64 libmplex2-2.1-0t64 libqt6test6 python3-fs rpcsvc-proto
libann0 libgavi-1 libkmlcore1t64 libpoppler147 libqt6xml6 python3-gpg samba-ad-dc
libarmadillo14 libgdal37 libkmlengine1t64 libportmidi0 libraw23t64 python3-pyqt6.sip samba-ad-provision
libavc1394-0 libgirepository-1.0-1 liblab-gamut1 libpython3.13-dev libwireshark18 python3.13 samba-dsdb-modules
libblosc2-4 libpgmep6t64 libmbedcrypto16 libqt6openglwidgets6 libwiretap15 python3.13-dev
libcdt5 libjs-jquery-ui libmjpegutils-2.1-0t64 libqt6sql6 libwsutil16 python3.13-minimal
libcgraph6 libjs-underscore libmpeg2encpp-2.1-0t64 libqt6sql6-sqlite libximg3 python3.13-venv
Use 'sudo apt autoremove' to remove them.
```

Figure 1: Directory Setup & Package Installations — Creating /SBT-DF203-Lab5 directory tree and installing Scapy, TShark, net-tools, and Wget.

```

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ls -lh evidence/arp.pcap
file evidence/arp.pcap
-rw----- 1 kali kali 1.1K Sep 14 16:25 evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
cat reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
cat reports/interfaces.txt
cat reports/routes.txt
cat reports/arp_table_initial.txt
lo                UNKNOWN          127.0.0.1/8  ::1/128
eth0              UP                192.168.199.128/24  fe80::8c84:5c9c:8758:3178/64
default via 192.168.199.2 dev eth0 proto dhcp src 192.168.199.128 metric 100
192.168.199.0/24 dev eth0 proto kernel scope link src 192.168.199.128 metric 100
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 STALE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE
lo                UNKNOWN          127.0.0.1/8  ::1/128
eth0              UP                192.168.199.128/24  fe80::8c84:5c9c:8758:3178/64
default via 192.168.199.2 dev eth0 proto dhcp src 192.168.199.128 metric 100
192.168.199.0/24 dev eth0 proto kernel scope link src 192.168.199.128 metric 100
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 STALE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ █

```

Figure 2: *arp.pcap Ingestion & Live Network Addressing Inspection* — Ingesting arp.pcap, computing SHA-256 hash, and reviewing ip address, routing, and neighbor table.

```

kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help

kali@kali: ~/SBT-DF203-Lab5 x kali@kali: ~/SBT-DF203-Lab5
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
echo "Interface: $IFACE"
echo "Gateway: $GATEWAY_IP"
Interface: eth0
Gateway: 192.168.199.2

(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo tshark -i $IFACE -f 'arp' -w evidence/normal_arp.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/normal_arp.pcapng") could not be opened: Permission denied.

(kali@kali)-[~/SBT-DF203-Lab5]
$

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
IFACE=eth0
tshark -i $IFACE -f 'arp' -w evidence/normal_arp.pcapng
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
4 ^C

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
cp --preserve=timestamp evidence/normal_arp.pcapng working/normal_arp_working.pcapng
sha256sum evidence/normal_arp.pcapng working/normal_arp_working.pcapng | tee reports/normal_arp_hashes.txt
cat reports/normal_arp_hashes.txt
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 evidence/normal_arp.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 working/normal_arp_working.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 evidence/normal_arp.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 working/normal_arp_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Figure 3: Baseline ARP Capture on Interface eth0 — Capturing live ARP resolution on eth0 to normal_arp.pcapng and recording SHA-256 hash.

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
cp --preserve=timestamp evidence/normal_arp.pcapng working/normal_arp_working.pcapng
sha256sum evidence/normal_arp.pcapng working/normal_arp_working.pcapng | tee reports/normal_arp_hashes.txt
cat reports/normal_arp_hashes.txt
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 evidence/normal_arp.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 working/normal_arp_working.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 evidence/normal_arp.pcapng
431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50 working/normal_arp_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_after_ping.txt
cat reports/arp_table_after_ping.txt
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 REACHABLE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 REACHABLE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Figure 4: Hash Preservation & Neighbor Table Post-Ping State — Verifying baseline SHA-256 hash and confirming gateway 192.168.199.2 is in REACHABLE state.

```
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab5 x kali@kali: ~/SBT-DF203-Lab5 x
(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
echo "Gateway: $GATEWAY_IP"
sudo ip neigh flush $GATEWAY_IP dev $IFACE
ping -c 2 $GATEWAY_IP
Gateway: 192.168.199.2
[sudo] password for kali:
PING 192.168.199.2 (192.168.199.2) 56(84) bytes of data.
64 bytes from 192.168.199.2: icmp_seq=1 ttl=128 time=3.30 ms
64 bytes from 192.168.199.2: icmp_seq=2 ttl=128 time=1.04 ms

— 192.168.199.2 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 1.037/2.169/3.301/1.132 ms
(kali@kali)-[~/SBT-DF203-Lab5]
$
```

Figure 5: *Neighbor Cache Flushing & Gateway Ping Test* — Flushing neighbor entry and confirming network reachability with 0% packet loss.

```

Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab5  kali@kali: ~/SBT-DF203-Lab5

(kali@kali)~/~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
tshark -r working/normal_arp_working.pcapng -Y 'arp' -T fields -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/normal_arp_fields.tsv
cat reports/normal_arp_fields.tsv
Warning: program compiled against libxml 215 using older 214
1 2026-09-16T18:10:40.397639416-0400 00:0c:29:f3:f7:44 00:50:56:ff:50:a0 1 192.168.199.128 00:0c:29:f3:f7:44 192.168.199.
2 00:00:00:00:00:00 00:0c:29:f3:f7:44 00:50:56:ff:50:a0 2 192.168.199.2 00:50:56:ff:50:a0 192.168.199.
128 00:0c:29:f3:f7:44
3 2026-09-16T18:10:58.177251976-0400 00:0c:29:f3:f7:44 ff:ff:ff:ff:ff:ff 1 192.168.199.128 00:0c:29:f3:f7:44 192.168.199.
2 00:00:00:00:00:00
4 2026-09-16T18:10:58.177678216-0400 00:50:56:ff:50:a0 00:0c:29:f3:f7:44 2 192.168.199.2 00:50:56:ff:50:a0 192.168.199.
128 00:0c:29:f3:f7:44
2026-09-16T18:10:40.397639416-0400 00:0c:29:f3:f7:44 00:50:56:ff:50:a0 1 192.168.199.128 00:0c:29:f3:f7:44 192.168.199.
2 00:00:00:00:00:00
2026-09-16T18:10:40.397937795-0400 00:50:56:ff:50:a0 00:0c:29:f3:f7:44 2 192.168.199.2 00:50:56:ff:50:a0 192.168.199.
128 00:0c:29:f3:f7:44
3 2026-09-16T18:10:58.177251976-0400 00:0c:29:f3:f7:44 ff:ff:ff:ff:ff:ff 1 192.168.199.128 00:0c:29:f3:f7:44 192.168.199.
2 00:00:00:00:00:00
4 2026-09-16T18:10:58.177678216-0400 00:50:56:ff:50:a0 00:0c:29:f3:f7:44 2 192.168.199.2 00:50:56:ff:50:a0 192.168.199.
128 00:0c:29:f3:f7:44

(kali@kali)~/~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
PCAP:working/arp_working.pcap
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.
proto_ipv4 -e arp.dst.hw_mac | tee reports/arp_replies.tsv
cat reports/arp_replies.tsv
Warning: program compiled against libxml 215 using older 214
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043799 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043799 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65

(kali@kali)~/~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
cat reports/ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65

```

Figure 6: Normal ARP Request/Reply Dissection & Opcode 2 Filtering — Extracting ARP request/reply frames and listing opcode 2 replies from arp_working.pcap.

```

(kali@kali)~/~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
cat reports/ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~/~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst==ff:ff:ff:ff:ff:ff' -T fields -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e et
h.dst | tee reports/unicast_arp_replies.tsv
cat reports/unicast_arp_replies.tsv
Warning: program compiled against libxml 215 using older 214

(kali@kali)~/~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==1' -T fields -e frame.number -e frame.time -e eth.src -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 |
tee reports/arp_requests.tsv
cat reports/arp_requests.tsv
Warning: program compiled against libxml 215 using older 214
1 2023-03-11T22:22:03.059513401-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1
2 2023-03-11T22:22:38.192015486-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199
3 2023-03-11T22:22:44.903581555-0500 00:50:56:86:cb:fc 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194
5 2023-03-11T22:22:49.943023192-0500 00:50:56:86:02:65 136.160.215.194 00:50:56:86:02:65 136.160.215.15
7 2023-03-11T22:23:03.059262685-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1
8 2023-03-11T22:23:06.191965551-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183
1 2023-03-11T22:22:03.059513401-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1
2 2023-03-11T22:22:38.192015486-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199
3 2023-03-11T22:22:44.903581555-0500 00:50:56:86:cb:fc 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194
5 2023-03-11T22:22:49.943023192-0500 00:50:56:86:02:65 136.160.215.194 00:50:56:86:02:65 136.160.215.15
7 2023-03-11T22:23:03.059262685-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1
8 2023-03-11T22:23:06.191965551-0500 00:1b:17:00:0a:30 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183

(kali@kali)~/~/SBT-DF203-Lab5]
$

```

Figure 7: IP-to-MAC Claims & Complete ARP Request Inventory — Sorting unique IP/MAC pairs and tabulating request frames (1, 2, 3, 5, 7, 8).

```

Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab5  kali@kali: ~/SBT-DF203-Lab5

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5
tshark -r working/arp_working.pcap -Y 'arp.opcode==2 && eth.src != arp.src.hw_mac' -T fields -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.src.
proto_ipv4 -e arp.src.hw_mac | tee reports/mac_mismatch_evidence.txt
cat reports/mac_mismatch_evidence.txt
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields -e frame.number -e frame.time_relative -e eth.src -e arp.src.proto_ipv4 -e arp.src.hw_ma
c -e arp.dst.proto_ipv4 | tee reports/poisoning_timeline.tsv
cat reports/poisoning_timeline.tsv
Warning: program compiled against libxml 215 using older 214
4      41.844610241      00:50:56:86:02:65      136.160.215.194      00:50:56:86:02:65      136.160.215.15
6      46.883530398      00:50:56:86:cb:fc      136.160.215.15      00:50:56:86:cb:fc      136.160.215.194
4      41.844610241      00:50:56:86:02:65      136.160.215.194      00:50:56:86:02:65      136.160.215.15
6      46.883530398      00:50:56:86:cb:fc      136.160.215.15      00:50:56:86:cb:fc      136.160.215.194

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==1 && arp.src.proto_ipv4 == arp.dst.proto_ipv4' -T fields -e frame.number -e arp.src.proto_ipv4 -e arp
.src.hw_mac | tee reports/gratuitous_arp.txt
cat reports/gratuitous_arp.txt
echo "(Empty = no gratuitous ARP, but ARP replies without requests are still suspicious)"
Warning: program compiled against libxml 215 using older 214
1      136.160.215.1      00:1b:17:00:0a:30
7      136.160.215.1      00:1b:17:00:0a:30
1      136.160.215.1      00:1b:17:00:0a:30
7      136.160.215.1      00:1b:17:00:0a:30
(Empty = no gratuitous ARP, but ARP replies without requests are still suspicious)

(kali@kali)-[~/SBT-DF203-Lab5]
$ cat > reports/arp_comparison.txt << 'EOF'
=== CLEAN ARP TABLE (Baseline) ===
EOF
cat reports/arp_table_initial.txt >> reports/arp_comparison.txt
echo "" >> reports/arp_comparison.txt
echo "=== POISONED CAPTURE CLAIMS ===" >> reports/arp_comparison.txt
cat reports/ip_mac_claims.txt >> reports/arp_comparison.txt
echo "" >> reports/arp_comparison.txt
echo "=== ANALYSIS ===" >> reports/arp_comparison.txt
echo "Legitimate gateway: 136.160.215.1 at 00:1b:17:00:0a:30" >> reports/arp_comparison.txt
echo "Poisoning indicator: eth.src != arp.src.hw_mac in frames 4 and 6" >> reports/arp_comparison.txt

```

Figure 8: Poisoning Timeline & Gratuitous ARP Identification — Filtering relative time offsets and identifying Frames 1 & 7 as gratuitous requests.

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ cat > reports/arp_comparison.txt << 'EOF'
=== CLEAN ARP TABLE (Baseline) ===
EOF
cat reports/arp_table_initial.txt >> reports/arp_comparison.txt
echo "" >> reports/arp_comparison.txt
echo "=== POISONED CAPTURE CLAIMS ===" >> reports/arp_comparison.txt
cat reports/ip_mac_claims.txt >> reports/arp_comparison.txt
echo "" >> reports/arp_comparison.txt
echo "=== ANALYSIS ===" >> reports/arp_comparison.txt
echo "Legitimate gateway: 136.160.215.1 at 00:1b:17:00:0a:30" >> reports/arp_comparison.txt
echo "Poisoning indicator: eth.src != arp.src.hw_mac in frames 4 and 6" >> reports/arp_comparison.txt
cat reports/arp_comparison.txt
=== CLEAN ARP TABLE (Baseline) ===
192.168.199.2 dev eth0 lladdr 00:50:56:ff:50:a0 STALE
192.168.199.254 dev eth0 lladdr 00:50:56:f5:93:70 STALE

=== POISONED CAPTURE CLAIMS ===
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

=== ANALYSIS ===
Legitimate gateway: 136.160.215.1 at 00:1b:17:00:0a:30
Poisoning indicator: eth.src != arp.src.hw_mac in frames 4 and 6

(kali@kali)-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]carpy' | tee reports/process_check.txt
cat reports/process_check.txt
echo "(Empty = no rogue process, clean)"
(Empty = no rogue process, clean)

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Figure 9: Baseline vs Capture Comparison & Process Tree Audit — Executing comparative analysis and running ps aux to verify no rogue Scapy processes exist.

```

(kali@kali)~/SBT-DF203-Lab5
$ cd ~/SBT-DF203-Lab5
tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields -e frame.number -e eth.src -e arp.src.hw_mac -e arp.src.proto_ipv4 2>/dev/null | python3 -c
"""
import sys
print('== ARP SPOOFING DETECTION ==')
print()
count = 0
for line in sys.stdin:
    parts = line.strip().split('\t')
    if len(parts) >= 4:
        frame, eth_src, arp_mac, ip = parts[0], parts[1], parts[2], parts[3]
        if eth_src.lower() != arp_mac.lower():
            count += 1
            print(f'MISMATCH - Frame {frame}')
            print(f' Ethernet Src: {eth_src}')
            print(f' ARP Payload MAC: {arp_mac}')
            print(f' Claimed IP: {ip}')
            print(f' Verdict: ARP SPOOFING INDICATOR')
            print()
print(f'Total mismatches found: {count}')
" | tee reports/mac_mismatch_evidence.txt
cat reports/mac_mismatch_evidence.txt
== ARP SPOOFING DETECTION ==

Total mismatches found: 0
== ARP SPOOFING DETECTION ==

Total mismatches found: 0

(kali@kali)~/SBT-DF203-Lab5
$ head -3 reports/arp_replies.tsv | cat -A
4^I1678591364.904123642^I00:50:56:86:02:65^I00:50:56:86:cb:fc^I136.160.215.194^I00:50:56:86:02:65^I136.160.215.15^I00:50:56:86:cb:fc$
6^I1678591369.943043799^I00:50:56:86:cb:fc^I00:50:56:86:02:65^I136.160.215.15^I00:50:56:86:cb:fc^I136.160.215.194^I00:50:56:86:02:65$

(kali@kali)~/SBT-DF203-Lab5
$

```

Figure 10: Python Classic Spoofing Mismatch Detection — Piping TShark fields to Python parser verifying 0 classic eth.src != arp.src.hw_mac mismatches.

```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab5 x kali@kali: ~/SBT-DF203-Lab5 x

(kali@kali)-[~/SBT-DF203-Lab5]
$ >....
for line in f:
    p = line.strip().split('\t')
    if len(p) >= 8:
        frame, eth_src, arp_mac, ip = p[0], p[2], p[5], p[4]
        if eth_src.lower() != arp_mac.lower():
            mismatch += 1
            print(f" MISMATCH Frame {frame}: eth.src={eth_src} != arp.src.hw_mac={arp_mac}")
print(f" Total classic mismatches: {mismatch}")
print()

print("— PART 2: Gratuitous ARP Requests (Gateway asking for itself) —")
import subprocess
out = subprocess.run(['tshark', '-r', 'working/arp_working.pcap',
    '-Y', 'arp.opcode==1 && arp.src.proto_ipv4 == arp.dst.proto_ipv4',
    '-T', 'fields', '-e', 'frame.number', '-e', 'arp.src.proto_ipv4', '-e', 'arp.src.hw_mac'],
    capture_output=True, text=True).stdout
grat_count = 0
for line in out.strip().split('\n'):
    if line:
        grat_count += 1
        print(f" GRATUITOUS Frame {line}")
print(f" Total gratuitous requests: {grat_count}")
print()

print("— PART 3: IP-to-MAC Claim Summary —")
with open('reports/ip_mac_claims.txt') as f:
    for line in f:
        print(f" {line.strip()}")
print()
print("=== SUMMARY ===")
print(f"Classic spoofing mismatches: {mismatch}")
print(f"Gratuitous ARP requests: {grat_count}")
print()
print("INTERPRETATION:")
print("This capture does NOT show classic eth.src/arp.src.hw_mac spoofing.")
print("The suspicious indicators are the gratuitous ARP requests from")
print("the gateway (136.160.215.1 asking for its own IP), which can signal")
print("an active poisoning attempt or gateway MAC re-assertion.")
```

Figure 11: *Comprehensive Python ARP Analysis Script* — Authoring automated diagnostic script evaluating classic mismatches, gratuitous ARPs, and claims.

```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab5 x kali@kali: ~/SBT-DF203-Lab5 x
cat reports/mac_mismatch_evidence.txt
== ARP POISONING DETECTION - Comprehensive Analysis ==

--- PART 1: eth.src vs arp.src.hw_mac (Classic Spoofing) ---
Total classic mismatches: 0

--- PART 2: Gratuitous ARP Requests (Gateway asking for itself) ---
GRATUITOUS Frame 1 136.160.215.1 00:1b:17:00:0a:30
GRATUITOUS Frame 7 136.160.215.1 00:1b:17:00:0a:30
Total gratuitous requests: 2

--- PART 3: IP-to-MAC Claim Summary ---
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

== SUMMARY ==
Classic spoofing mismatches: 0
Gratuitous ARP requests: 2

INTERPRETATION:
This capture does NOT show classic eth.src/arp.src.hw_mac spoofing.
The suspicious indicators are the gratuitous ARP requests from
the gateway (136.160.215.1 asking for its own IP), which can signal
an active poisoning attempt or gateway MAC re-assertion.
== ARP POISONING DETECTION - Comprehensive Analysis ==

--- PART 1: eth.src vs arp.src.hw_mac (Classic Spoofing) ---
Total classic mismatches: 0

--- PART 2: Gratuitous ARP Requests (Gateway asking for itself) ---
GRATUITOUS Frame 1 136.160.215.1 00:1b:17:00:0a:30
GRATUITOUS Frame 7 136.160.215.1 00:1b:17:00:0a:30
Total gratuitous requests: 2

--- PART 3: IP-to-MAC Claim Summary ---
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

== SUMMARY ==
Classic spoofing mismatches: 0
Gratuitous ARP requests: 2
```

Figure 12: *Final Comprehensive Analysis Output* — Execution output confirming 0 classic mismatches, 2 gratuitous ARPs from gateway 136.160.215.1, and interpretation.

10. Conclusions

This lab demonstrated the forensic analysis of ARP traffic for poisoning indicators. The supplied capture contained gratuitous ARP requests from the gateway — a suspicious behavior — but no classic Ethernet/ARP MAC mismatch. This highlights the importance of understanding multiple indicators and not relying solely on any single pattern. All evidence was preserved with matching SHA-256 hashes.

11. Appendix – Commands and Hashes

```
# Baseline Network State
ip -br address
ip route
ip neigh show

# Flush & Ping Gateway
sudo ip neigh flush 192.168.199.2 dev eth0
ping -c 2 192.168.199.2

# Live Baseline Capture
tshark -i eth0 -f 'arp' -w evidence/normal_arp.pcapng

# Forensic Dissection of Evidence
tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields ...
tshark -r working/arp_working.pcap -Y 'arp.opcode==1 && arp.src.proto_ipv4 == arp.dst.proto_ipv4' ...

# Cryptographic Hash Verification
sha256sum evidence/*.pcap* working/*.pcap*
```

Evidence Integrity: Cryptographic Hash Digest Preservation:

- **arp.pcap Hash:** 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
- **arp_working.pcap Hash:** 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
- **normal_arp.pcapng Hash:** 431c563969b81228b91aeb163861b156c0f0196e74bdd264bf0d0a97bf971d50

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Student Signature: Almustapha Yusuf

Date: 16/09/2026